

Plataforma de Envio em Massa WhatsApp

Pesquisa competitiva, arquitetura tecnica, anti-banimento completo e implementacao integrada ao ZEHLA Brain + LIS para 10.000+ contatos de pousadas.

ZEHLA BLAST • 2026 • v1.0

WaSeller + Zap Responder + Evolution API | 3 modulos Prisma | BullMQ + Redis | Pool de instancias | Throttler inteligente | Integracao ZEHLA Brain (score + cluster)

PESQUISA + ARQUITETURA + CODIGO

Sumario

1. Pesquisa Competitiva — Anatomia dos Concorrentes	2
1.1. WaSeller.com.br — Extensao Chrome	2
1.2. Zap Responder — SaaS Dual-Mode	3
1.3. Evolution API — Biblioteca Open-Source	3
1.4. Comparativo dos Concorrentes	4
2. Tecnicas Anti-Banimento — Manual Completo	4
2.1. Protocolo de Aquecimento de Numero (Chip Warming)	5
2.2. Tecnicas de Throttling e Ritmo Humano	5
2.3. Sistema de Opt-Out e Compliance	5
2.4. Rotacao de Numeros e Pool de Instancias	6
3. Arquitetura do ZEHLA Blast	6
3.1. Diagrama de Arquitetura	6
3.2. Componentes do Sistema	7
4. Prisma Schema — Modelos de Dados	7
5. Blast Engine — Core de Envio	10
5.1. Evolution API Client	10
5.2. Blast Queue Setup	12
5.3. Campaign Sender Service	12
5.4. ZEHLA Brain Integration (Tracking)	14
6. API Routes — Endpoints do ZEHLA Blast	15
7. Deploy — Docker Compose Completo	15
8. Fluxo Completo End-to-End	17
9. ZCC Dashboard — Interface do Blast	18
10. Estrutura de Arquivos — Modulo Blast	18
11. Varias Tecnicas Recomendadas	19

1. Pesquisa Competitiva — Anatomia dos Concorrentes

Esta secao apresenta a anatomia detalhada das duas plataformas referenciais para envio de mensagens WhatsApp em massa no mercado brasileiro. A pesquisa foi conduzida via web scraping, analise de codigo fonte (Chrome Web Store, npm), documentacao de help center e conteudo de blogs. O objetivo e extrair os conceitos, workflows, tecnicas anti-ban e arquiteturas que serao adaptados para a ferramenta ZEHLA Blast.

1.1. WaSeller.com.br — Extensao Chrome

WaSeller e uma extensao do Google Chrome que se sobrepoe ao WhatsApp Web transformando-o em uma ferramenta de CRM e vendas. NAO e um SaaS, NAO usa API oficial e NAO requer login externo. Toda a funcionalidade roda client-side via injecao de JavaScript no DOM do WhatsApp Web. Possui 100.000+ usuarios ativos e nota 4.8 estrelas na Chrome Web Store. O preco e R\$ 397/ano para acesso ilimitado.

Feature	Descricao	Tecnologia
Envio em Massa	Texto, audio, imagem, video, documentos	DOM manipulation
Funil de Mensagem	Sequencias pre-construidas com 1 clique	Timer + DOM queue
Chatbot Flows	Fluxos interativos com opcoes clicaveis	State machine client-side
Auto-Responder	Respostas automaticas simples	Message listener + timer
Agendamento	Mensagens programadas para horarios futuros	localStorage + setTimeout
Import CSV	Contatos com higienizacao DDI/DDD	PapaParse client-side
Kanban CRM	Pipeline visual de vendas no sidebar	Drag-drop injected UI
Tags / Etiquetas	Segmentacao ilimitada de contatos	IndexedDB local
Variaveis	{nome}, {produto}, {oferta}, {data}	Template engine regex
Quick Replies	Mensagens pre-salvas com 1 clique	Hotkey + DOM injection
AI Assistant	Escrita, traducao, resumo	API call a LLM externo
Webhook	Notificacoes para endpoints externos	fetch() POST premium

Tabela 1: Features completas do WaSeller

WaSeller — Como funciona o envio em massa: O fluxo e: (1) Importar CSV com contatos, (2) Higienizar DDI/DDD e mesclar duplicatas, (3) Compormensagem com variaveis ({nome}, {produto}), (4) Definir lotes e horarios, (5) Enviar teste para 10-50 contatos, (6) Lancar campanha e monitorar taxa de resposta em tempo real. Tudo via DOM manipulation do WhatsApp Web.

1.2. Zap Responder — SaaS Dual-Mode

Zap Responder é uma plataforma SaaS brasileira completa fundada por Afonso Martins. Centraliza WhatsApp, Instagram Direct e Facebook Messenger em um unico dashboard. Opera em DOIS modos de conexao: (A) QR Code via Baileys (gratuito, nao-oficial) e (B) Meta Official Cloud API (pago, compliant). Possui builder visual de chatbot, CRM completo, sistema de campanhas com throttling, agentes IA (SDR-RAG), e Chrome Extension. Possui pacote npm proprio: @zapresponder/baileys v6.7.10.

Feature	Descricao	Tecnologia
CRM + Leads	Pipeline completo com dashboard KPIs	Web app (Bubble.io)
Campanhas em Massa	Wizard 3 passos com throttling completo	Baileys / Meta API
Builder de Chatbot	Fluxos visuais com variaveis e handoff	Visual flow editor
Agente IA (SDR-RAG)	Atendente virtual com RAG + base conhecimento	OpenAI + Dify
Multi-Canal	WhatsApp + Instagram + Messenger unificados	Multi-adapter API
Departamentos	Multi-numero com sessoes independentes	Instance isolation
Agendamento	Dias, horarios min/max, delay aleatorio	Cron + BullMQ-like queue
Import CSV	Contatos com grupos e segmentacao	Server-side parser
Opt-out #sair	Contatos digitam #sair para sair	Message filter + DB flag
Templates Meta	Templates aprovados pela Meta	Cloud API HSM
Avaliacoes	Rating de satisfacao pos-atendimento	Collect + analytics
Relatorios	Dashboards com metricas de performance	Chart.js / Recharts

Tabela 2: Features completas do Zap Responder

1.3. Evolution API — Biblioteca Open-Source

Evolution API é a plataforma open-source de referencia para integracao WhatsApp (Apache 2.0, 6.8k stars no GitHub). Fornece uma REST API completa que abstrai a complexidade do protocolo WhatsApp Web via Baileys/Whatsmeow. Suporta multi-instancia (numeros ilimitados), webhooks em tempo real, integracoes com Typebot, Chatwoot, Dify, OpenAI e N8N. É a base tecnica recomendada para a implementacao do ZEHLA Blast, combinada com a Meta Official API para escala.

Feature	Descricao	Tecnologia
REST API	Endpoints para todas as operacoes	Express.js (porta 8080)
Multi-Instancia	Numeros ilimitados, isolamento total	Multi-tenant architecture
Baileys + Whatsmeow	Conexao via WebSocket (QR ou pairing code)	Protocolo binario WA Web

Cloud API Meta	Conexao oficial para producao	Meta Business API
Webhooks	Receber mensagens, status, conexoes	HTTP POST + Kafka/RabbitMQ/SQS
Media Storage	S3 ou Minio para imagens/docs	Pre-signed URLs
Chatbot Builder	Typebot integration (visual no-code)	/typebot/create/{instance}
AI Agents	OpenAI Whisper + GPT + Dify	/chat/ai/{instance}
N8N Automation	Workflows visuais complexos	Webhook event stream
Docker Deploy	docker-compose com tudo incluso	PostgreSQL + Redis + S3

Tabela 3: Features do Evolution API

1.4. Comparativo dos Concorrentes

Criterio	WaSeller	Zap Responder	Evolution API	ZEHLA Blast (plano)
Tipo	Chrome Extension	SaaS Plataforma	API Open-Source	Modulo Integrado
Conexao WA	DOM (WhatsApp Web)	Baileys + Meta API	Baileys + Meta API	Evolution API + Meta
Anti-Ban	Ritmo humano	Meta API + delays	Delays + limites	Warm-up + delays + opt-out
CRM	Kanbas basico	Completo com KPIs	Nao (precisa Chatwoot)	ZEHLA LIS + Brain
Chatbot	Flows simples	Visual builder + IA	Typebot + OpenAI	Flows + ZEHLA Brain IA
Campanhas	Basico com CSV	Wizard 3 passos	API direta	Campanhas + LIS integration
Multi-Numero	Nao (1 browser)	Sim (departamentos)	Sim (multi-instance)	Sim (rotation pool)
Preco	R\$ 397/ano	R\$ 29-89/mes	Gratuito	Custo Zero (proprio)
Codigo Aberto	Nao	Nao	Sim (Apache 2.0)	Sim (interno)
Score Reclame Aqui	6.6/10	N/A	N/A	N/A (novo)

Tabela 4: Comparativo competitivo

2. Tecnicas Anti-Banimento — Manual Completo

Esta secao consolida todas as tecnicas de protecao contra banimento identificadas na pesquisa dos dois concorrentes e da documentacao do Evolution API e Baileys. O WhatsApp bane cerca de 8 milhoes de contas por ano na India so (2025). As tecnicas abaixo reduzem drasticamente o risco mas nao o eliminam completamente. A unica forma 100% segura e usar a Meta Oficial API.

2.1. Protocolo de Aquecimento de Numero (Chip Warming)

Numeros novos ou recém-conectados tem limites muito baixos. O protocolo de aquecimento gradual aumenta esses limites ao longo de 30-90 dias. O ZEHLA Blast implementara esse protocolo automaticamente, recusando-se a enviar acima do limite diario configurado para cada instancia.

Periodo	Limite Diario	Orientacao
Dias 1-3	10-20 msgs/dia	Enviar para contatos conhecidos, trocar mensagens organicas
Dias 4-7	30-50 msgs/dia	Iniciar conversas bidirecionais, responder mensagens recebidas
Dias 8-14	50-100 msgs/dia	Primeiros envios de campanha, apenas contatos opt-in
Dias 15-30	100-200 msgs/dia	Campanhas maiores, monitorar taxa de bloqueio (< 2%)
Dias 31-60	200-400 msgs/dia	Operacao normal com throttling agressivo
Dias 61-90	400-800 msgs/dia	Operacao plena, delays de 20-30s entre mensagens
90+ dias	800-1000 msgs/dia	Limite maximo, manter monitoramento continuo

Tabela 5: Protocolo de aquecimento de numero

2.2. Tecnicas de Throttling e Ritmo Humano

O throttling controla a velocidade de envio para simular comportamento humano. O ZEHLA Blast implementa multiplas camadas de throttling: delay por mensagem, delay por lote, pausa entre lotes, e jitter aleatorio. Cada camada e configuravel por campanha e por instancia.

- **Delay por mensagem:** 20-60 segundos entre cada mensagem, com jitter de +0 a +30s aleatorio
- **Lotes:** 20-30 mensagens por lote, seguido de pausa de 10-15 minutos
- **Horarios:** Enviar apenas entre 08:00-20:00 no fuso horario do destinatario
- **Dias:** Segunda a sexta apenas (Domingo nunca para campanhas frias)
- **Personalizacao:** Variar tom, tamanho e estrutura das mensagens
- **Bidirecionalidade:** Responder mensagens recebidas antes de enviar novas
- **Engajamento:** Se > 5% dos contatos bloqueiam, pausar imediatamente

2.3. Sistema de Opt-Out e Compliance

Todo lead que receber mensagens do ZEHLA Blast deve ter a opcao de sair a qualquer momento. O sistema implementa opt-out de 3 formas: (1) a mensagem inclui instrucoes claras de como parar ("responda SAIR"), (2) o ZEHLA Brain detecta a resposta e marca o lead como opted-out, e (3) o filtro de campanha exclui automaticamente contatos opted-out. Alem disso, todo contato importado deve ter registro de consentimento (data, fonte, metodo de opt-in).

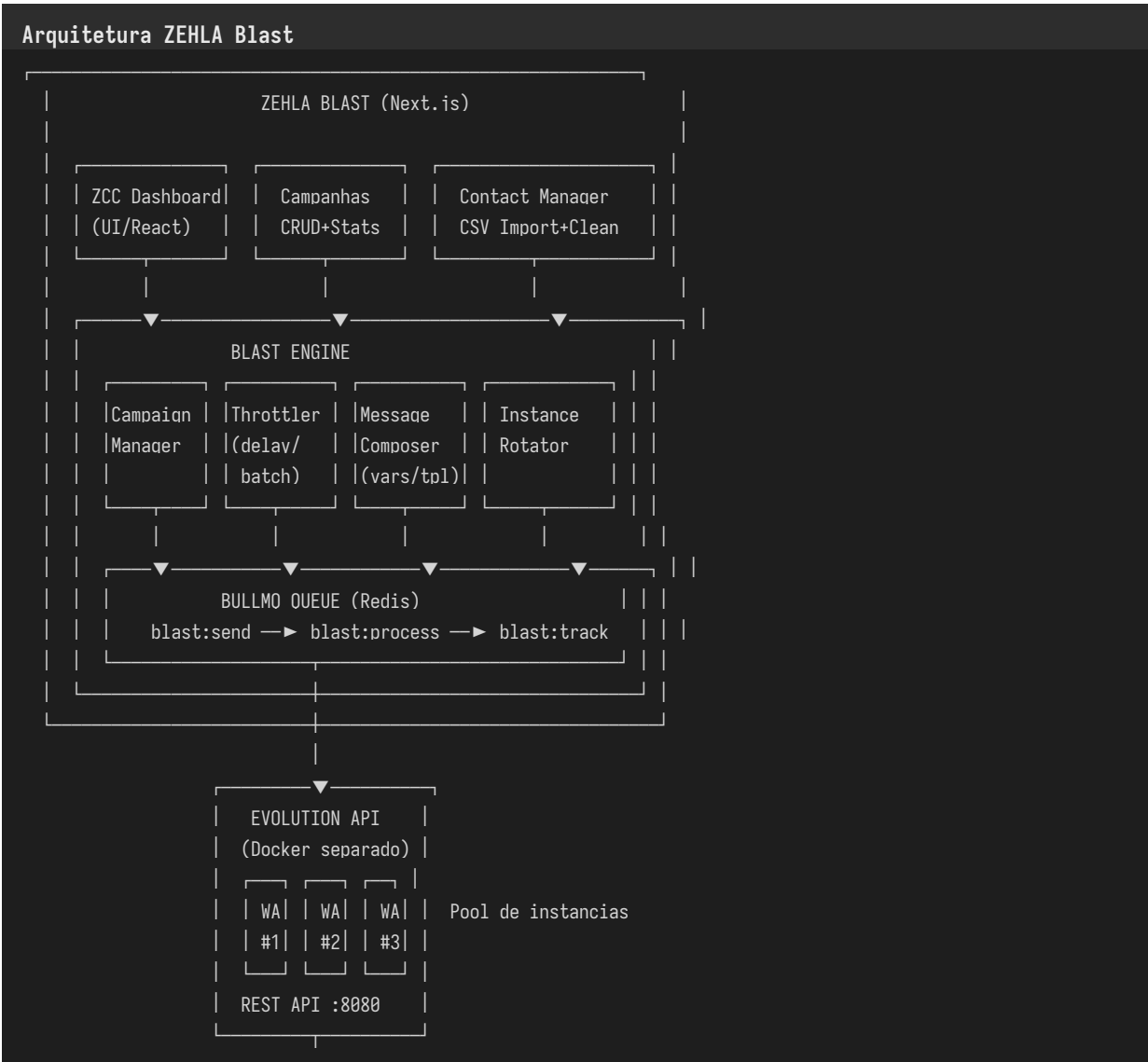
2.4. Rotacao de Numeros e Pool de Instancias

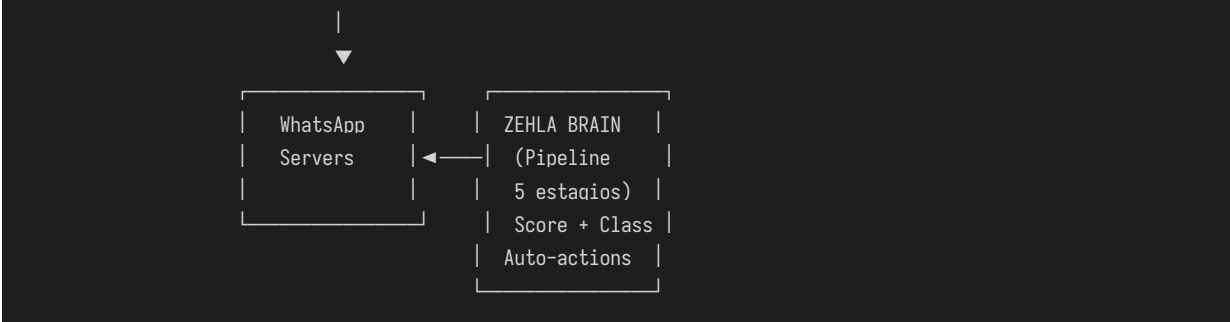
Para distribuir a carga e minimizar o risco, o ZEHLA Blast opera com um pool de instancias WhatsApp (3-5 numeros recomendados para 10.000 contatos). O sistema rotaciona automaticamente: cada instancia envia no maximo N mensagens por hora, depois o balanceador direciona para a proxima instancia. Se uma instancia recebe alerta do WhatsApp (qualidade baixando), ela e automaticamente pausada e as mensagens sao redistribuidas para as demais.

3. Arquitetura do ZEHLA Blast

O ZEHLA Blast e um modulo integrado ao ecossistema ZEHLA que combina as melhores praticas dos concorrentes pesquisados com a inteligencia do ZEHLA Brain e a gestao de leads do LIS. A arquitetura segue o principio de separacao de concerns: o Evolution API cuida da conexao WhatsApp, o ZEHLA Blast gerencia campanhas e throttling, e o ZEHLA Brain rastreia todas as interacoes para score e classificacao.

3.1. Diagrama de Arquitetura





3.2. Componentes do Sistema

Componente	Descricao	Tecnologia
ZCC Dashboard	Interface React com listagem de campanhas, metricas em tempo real e configuracoes de throttling	React + Recharts + Zustand
Campaign Manager	CRUD de campanhas com status (rascunho, ativa, pausada, concluida), filtros e agendamento	Prisma + Next.js API Routes
Contact Manager	Import CSV com higienizacao DDI/DDD, validacao, deduplicacao e segmentacao	PapaParse + Prisma
Message Composer	Editor de templates com variaveis ({nome}, {pousada}, {cidade}), preview e teste	Template Engine + React
Throttler Engine	Controle de ritmo: delay por msg, lote, pausa, jitter, horarios, dias	BullMQ + Redis + Cron
BullMQ Queue	Filas para envio (blast:send), processamento (blast:process) e tracking (blast:track)	BullMQ + Redis
Instance Rotator	Balanceamento de carga entre instancias WhatsApp (pool de 3-5 numeros)	Round-robin + health check
Evolution API Client	Cliente REST para conexao WhatsApp via Baileys/Meta API	fetch + WebSocket
Webhook Receiver	Recebe respostas dos leads e direciona para o ZEHLA Brain	Next.js API Route
ZEHLA Brain Integration	Track de eventos (whatsapp_open, whatsapp_reply) para score e classificacao	BullMQ + Prisma

Tabela 6: Componentes do ZEHLA Blast

4. Prisma Schema — Modelos de Dados

O schema do ZEHLA Blast adiciona 4 modelos ao Prisma existente: BlastCampaign (campanhas), BlastMessage (mensagens individuais), BlastInstance (instancias WhatsApp do pool), e BlastContact (contatos importados com consentimento). Esses modelos se integram com o Lead e Event existentes do ZEHLA LIS para rastreabilidade completa.

prisma/schema.prisma — BlastCampaign

```
model BlastCampaign {
  id          String    @id @default(cuid())
  name        String    // Nome da campanha
  status       String    @default("draft")
  // draft, scheduled, active, paused.
  // completed, failed
  // Content
  messageTemplate String // Template com {variaveis}
  mediaUrl     String?   // Imagem/video/documento
  mediaType    String?   // image, video, document, audio
  // Targeting
  contactGroup String    // Grupo de contatos
  totalContacts Int      @default(0)
  sentCount     Int      @default(0)
  deliveredCount Int     @default(0)
  readCount     Int      @default(0)
  repliedCount  Int      @default(0)
  failedCount   Int      @default(0)
  optedOutCount Int     @default(0)
  // Scheduling
  scheduledAt   DateTime? // Agendamento
  startedAt     DateTime?
  completedAt   DateTime?
  // Throttling config (JSON)
  config        Json?     // delayMsg, batchSize,
  // batchPause, jitter, hours, days
  // Instance pool
  instanceIds   String[]  // IDs das instancias Evolution API
  // Relations
  messages      BlastMessage[]
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
  @@map("blast_campaigns")
}
```

prisma/schema.prisma — BlastMessage

```
model BlastMessage {
  id          String    @id @default(cuid())
  campaignId   String
  campaign     BlastCampaign @relation(
    fields: [campaignId], references: [id])
  contactPhone String    // 5511999999999
  contactName  String?
  leadId       String?   // Link com Lead do LIS
  // Status pipeline
  status       String    @default("pending")
  // pending, queued, sent, delivered.
  // read, replied, failed, opted_out
  // Personalized content
  renderedMessage String? // Template preenchido
  mediaUrl     String?
  // Instance used
  instanceId   String?
  // Tracking
  sentAt       DateTime?
```

```

deliveredAt  DateTime?
readAt       DateTime?
repliedAt    DateTime?
failedReason String?
optedOutAt   DateTime?
// ZEHLA Brain event tracking
brainEventId String? // Link com LeadEvent
createdAt    DateTime @default(now())
@@index([campaignId])
@@index([status])
@@index([contactPhone])
@@map("blast_messages")
}

```

prisma/schema.prisma — BlastInstance

```

model BlastInstance {
  id          String  @id @default(cuid())
  name        String           // Nome amigavel
  phone       String           // 5511999999999
  evolutionUrl String          // URL do Evolution API
  apiKey      String           // API key da instancia
  instanceName String          // Nome no Evolution API
  status       String  @default("disconnected")
  // connected, disconnected, banned, paused
  // Warmup tracking
  createdAt    DateTime @default(now()) // Data de criacao
  warmupStage  Int       @default(0)    // Dias de aquecimento
  dailyLimit   Int       @default(20)    // Limite diario atual
  hourlyLimit  Int       @default(10)    // Limite por hora
  sentToday    Int       @default(0)     // Enviadas hoje
  sentThisHour Int       @default(0)     // Enviadas esta hora
  // Health
  lastError    String?
  lastErrorAt  DateTime?
  bannedAt     DateTime?
  qualityScore Float?           // 0-1
  campaigns    BlastCampaign[]
  @@unique([phone])
  @@map("blast_instances")
}

```

prisma/schema.prisma — BlastContact

```

model BlastContact {
  id          String  @id @default(cuid())
  phone       String           // 5511999999999
  name        String?
  email       String?
  pousadaName String?
  city        String?
  state       String?
  group       String  @default("geral")
  // Consent
  optIn       Boolean  @default(false)
  optInSource String?   // csv_import, form, manual
  optInDate   DateTime?
  optedOut    Boolean  @default(false)
}

```

```

    optedOutAt    DateTime?
    optedOutReason String?
    // Link with LIS
    leadId        String?           // Link com Lead do LIS
    createdAt     DateTime @default(now())
    updatedAt     DateTime @updatedAt
    @@unique([phone])
    @@index([group])
    @@index([optedOut])
    @@map("blast_contacts")
  }

```

5. Blast Engine — Core de Envio

O Blast Engine é o núcleo da operação. Ele gerencia a fila de envio via BullMQ, controla o throttling com delays e lotes, rotaciona entre instâncias do pool, e integra com o ZEHLA Brain para tracking de todas as interações. A arquitetura usa 3 filas dedicadas: blast:send (enfileiramento), blast:process (envio real), e blast:track (pos-envio).

5.1. Evolution API Client

```

lib/evolution-client.ts

// lib/evolution-client.ts — Cliente Evolution API
import { BlastInstance } from '@prisma/client';

interface EvolutionMessage {
  number: string;
  text?: string;
  mediaUrl?: string;
  mediaType?: string;
  delay?: number;
}

export class EvolutionClient {
  private baseUrl: string;
  private apiKey: string;
  constructor(instance: BlastInstance) {
    this.baseUrl = instance.evolutionUrl;
    this.apiKey = instance.apiKey;
  }

  private async request(
    method: string, path: string, body?: any
  ) {
    const url = `${this.baseUrl}${path}`;
    const opts: any = {
      method,
      headers: {
        'Content-Type': 'application/json',
        'apikey': this.apiKey,
      },
    };
    if (body) opts.body = JSON.stringify(body);
    const res = await fetch(url, opts);
    if (!res.ok) {

```

```

    const err = await res.text();
    throw new Error(
      `Evolution API ${res.status}: ${err}`
    );
  }
  return res.json();
}
async checkConnection(): Promise<boolean> {
  try {
    const data = await this.request(
      'GET',
      `/instance/connectionState/${this.instanceName}`
    );
    return data.state === 'connected';
  } catch { return false; }
}
async sendText(
  number: string, text: string, delay = 30
) {
  return this.request('POST',
    `/message/sendText/${this.instanceName}`, {
      number, text, delay
    });
}
async sendMedia(
  number: string,
  mediaUrl: string,
  mediaType: string,
  caption?: string,
  delay = 30
) {
  const endpoint = {
    image: 'sendImage',
    video: 'sendVideo',
    document: 'sendDocument',
    audio: 'sendAudio',
  }[mediaType] || 'sendImage';
  return this.request('POST',
    `/message/${endpoint}/${this.instanceName}`, {
      number, mediatype: mediaType,
      mediaUrl, caption, delay
    });
}
async getOrCode(): Promise<string> {
  const data = await this.request('GET',
    `/instance/connect/${this.instanceName}`);
  return data.base64;
}
}

```

5.2. Blast Queue Setup

lib/blast-queues.ts

```
// lib/blast-queues.ts — Filas BullMQ do Blast Engine
import { Queue, Worker, Job } from 'bullmq';
import { redis } from './redis';
export const blastSendQueue = new Queue(
  'blast:send', { connection: redis }
);
export const blastProcessQueue = new Queue(
  'blast:process', { connection: redis }
);
export const blastTrackQueue = new Queue(
  'blast:track', { connection: redis }
);
// Worker de envio (1 concurrent para respeitar delays)
export const blastProcessWorker = new Worker(
  'blast:process',
  async (job: Job) => {
    const { messageId, phoneNumber, content,
      instanceId, delay } = job.data;
    // ... send via Evolution API
    // ... wait delay ms
    // ... update message status
  },
  {
    connection: redis,
    concurrency: 1, // 1 por instancia
    limiter: { max: 30, duration: 3600000 },
    // 30 msqs por hora por instancia
  }
);
```

5.3. Campaign Sender Service

services/campaign-sender.ts

```
// services/campaign-sender.ts
import { prisma } from '@lib/prisma';
import { blastProcessQueue } from '@lib/blast-queues';
import { EvolutionClient } from '@lib/evolution-client';
export async function launchCampaign(
  campaignId: string
) {
  const campaign = await prisma.blastCampaign
    .findUniqueOrThrow({
      where: { id: campaignId },
      include: { messages: true },
    });
  // Config de throttling
  const config = campaign.config || {};
  const delayMsg = config.delayMsg || 30000; // 30s default
  const batchSize = config.batchSize || 25;
  const batchPause = config.batchPause || 600000; // 10 min
  const jitter = config.jitter || 15000; // 15s random
  // Buscar instancias ativas
```

```

const instances = await prisma.blastInstance
  .findMany({
    where: { status: 'connected' },
  });
if (instances.length === 0) {
  throw new Error('Nenhuma instancia conectada');
}
// Marcar campanha como ativa
await prisma.blastCampaign.update({
  where: { id: campaignId },
  data: {
    status: 'active',
    startedAt: new Date(),
  },
});
// Buscar mensagens pendentes
const messages = await prisma.blastMessage
  .findMany({
    where: {
      campaignId,
      status: 'pending',
    },
    orderBy: { createdAt: 'asc' },
  });
// Distribuir entre instancias (round-robin)
let instanceIdx = 0;
for (const msg of messages) {
  const instance = instances[
    instanceIdx % instances.length
  ];
  instanceIdx++;
  const actualDelay =
    delayMsg + Math.random() * jitter;
  await blastProcessQueue.add(
    'send-message', {
      messageId: msg.id,
      campaignId,
      phoneNumber: msg.contactPhone,
      content: {
        text: msg.renderedMessage,
        mediaUrl: msg.mediaUrl,
        mediaType: campaign.mediaType,
      },
      instanceId: instance.id,
      delay: actualDelay,
    },
    {
      attempts: 3,
      backoff: {
        type: 'exponential',
        delay: 5000,
      },
      delay: actualDelay, // BullMQ delay
    }
  );
}
console.log(

```

```

    `[Blast] Campanha ${campaign.name}: ` +
    `${messages.length} msqs enfileiradas ` +
    `em ${instances.length} instancias`
  );
}

```

5.4. ZEHLA Brain Integration (Tracking)

Toda mensagem enviada pelo ZEHLA Blast gera eventos no ZEHLA Brain. Quando um lead responde a uma mensagem de campanha, o webhook receiver detecta a resposta, cria um LeadEvent do tipo whatsapp_reply (+30 pontos), e o pipeline de classificacao recalcula o score e cluster do lead. Isso permite que o Brain saiba exatamente quais leads estao engajando com as campanhas.

services/blast-brain-tracker.ts

```

// services/blast-brain-tracker.ts
// Toda mensagem enviada gera evento no ZEHLA Brain
import { prisma } from '@lib/prisma';
export async function trackBlastEvent(data: {
  messageId: string;
  leadId: string;
  eventType: 'whatsapp_open' | 'whatsapp_reply';
  campaignId: string;
}) {
  // Criar evento no ZEHLA Brain pipeline
  await fetch('/api/events/track', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      email: '', // lookup by leadId
      eventType: data.eventType,
      eventSource: 'zehla_blast',
      metadata: {
        messageId: data.messageId,
        campaignId: data.campaignId,
        blastCampaign: true,
      },
    }),
  });
  // Atualizar mensagem
  await prisma.blastMessage.update({
    where: { id: data.messageId },
    data: {
      status: data.eventType === 'whatsapp_reply'
        ? 'replied'
        : 'delivered',
      repliedAt: data.eventType === 'whatsapp_reply'
        ? new Date()
        : undefined,
    },
  });
  // Atualizar contadores da campanha
  const field = data.eventType === 'whatsapp_reply'

```

```

    ? 'repliedCount' : 'deliveredCount';
    await prisma.blastCampaign.update({
      where: { id: data.campaignId },
      data: { [field]: { increment: 1 } },
    });
  }
}

```

6. API Routes — Endpoints do ZEHLA Blast

Endpoint	Metodo	Funcao
/api/blast/campaigns	GET+POST	Listar e criar campanhas
/api/blast/campaigns/[id]	GET+PATCH+DELETE	Detalhe, atualizar e deletar campanha
/api/blast/campaigns/[id]/launch	POST	Iniciar campanha (enfileirar envio)
/api/blast/campaigns/[id]/pause	POST	Pausar campanha ativa
/api/blast/campaigns/[id]/resume	POST	Retomar campanha pausada
/api/blast/contacts	GET+POST	Listar e adicionar contatos
/api/blast/contacts/import	POST	Import CSV (multipart form)
/api/blast/contacts/export	GET	Exportar contatos (CSV)
/api/blast/instances	GET+POST	Listar e registrar instancias
/api/blast/instances/[id]/qr	GET	Obter QR code para conexao
/api/blast/instances/[id]/status	GET	Verificar status da instancia
/api/blast/webhook	POST	Webhook receiver (respostas dos leads)
/api/blast/templates	GET+POST	Listar e criar templates de mensagem
/api/blast/stats	GET	Metricas gerais (enviadas, lidas, respondidas)
/api/blast/health	GET	Health check do sistema Blast

Tabela 7: Endpoints API do ZEHLA Blast

7. Deploy — Docker Compose Completo

O deploy do ZEHLA Blast requer dois componentes: o Evolution API (Docker separado) e o modulo integrado ao projeto Next.js. O docker-compose abaixo provisiona tudo automaticamente: Evolution API com PostgreSQL e Redis, o servidor Next.js, e o BullMQ Board para monitoramento visual das filas em tempo real.

```
docker-compose.yml
```



```
# docker-compose.yml — ZEHLA Blast + Evolution API
version: '3.8'
services:
  # Evolution API (WhatsApp connection)
  evolution-api:
    image: atendai/evolution-api:latest
    ports:
      - "8080:8080"
    environment:
      - SERVER_PORT=8080
      - DATABASE_TYPE=postgresql
      - DATABASE_URL=postgres://evo:evo_pass@postgres-evo:5432/evolution
      - JWT_SECRET=${JWT_SECRET}
      - WHATSAPP_TYPE=bailevs
      - TYPEBOT_ENABLED=false
      - CHATWOOT_ENABLED=false
    depends_on:
      - postgres-evo
    restart: unless-stopped
  postgres-evo:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: evo
      POSTGRES_PASSWORD: evo_pass
      POSTGRES_DB: evolution
    volumes:
      - evo_data:/var/lib/postgresql/data
  # ZEHLA Next.js App
  smarthotel:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://zehla:zehla_pass@postgres:5432/zehla_db
      - REDIS_URL=redis://redis:6379
      - EVOLUTION_API_URL=http://evolution-api:8080
      - EVOLUTION_API_KEY=${EVO_API_KEY}
    depends_on:
      - postgres
      - redis
      - evolution-api
    restart: unless-stopped
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: zehla
      POSTGRES_PASSWORD: zehla_pass
      POSTGRES_DB: zehla_db
    volumes:
      - zehla_data:/var/lib/postgresql/data
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    command: >
      redis-server
      --appendonly yes
```

```

--maxmemory 512mb
--maxmemory-policy allkeys-lru
volumes:
  - redis_data:/data
bullmq-board:
  image: dockersamples/bullmq-board
  ports:
    - "3001:3000"
  environment:
    - REDIS_HOST=redis
  depends_on:
    - redis
volumes:
  evo_data:
  zehla_data:
  redis_data:

```

8. Fluxo Completo End-to-End

O fluxo abaixo descreve o ciclo completo de uma campanha de envio em massa, desde a importacao dos contatos ate a classificacao automatica dos leads que responderam pelo ZEHLA Brain.

Eta pa	Acao	Sistema
1	Importar CSV com 10.000 contatos de pousadas	Contact Manager
2	Higienizar DDI/DDD, deduplicar, segmentar por UF	Contact Manager
3	Criar campanha com template + variaveis	Campaign Manager
4	Configurar throttling (delay 30s, lote 25, pausa 10min)	Throttler
5	Conectar 3-5 instancias WhatsApp via QR Code	Evolution API
6	Verificar aquecimento dos numeros (verificar warmup stage)	Instance Manager
7	Enviar teste para 10 contatos selecionados	Blast Engine
8	Lancar campanha (enfileirar 10.000 mensagens)	BullMQ Queue
9	Blast Process Worker envia msg por msg com delay	Worker + Evolution API
10	Rotacionar entre instancias (round-robin)	Instance Rotator
11	Lead recebe mensagem e abre WhatsApp	WhatsApp + Webhook
12	Webhook detecta abertura -> whatsapp_open (+25 pts)	ZEHLA Brain
13	Lead responde -> whatsapp_reply (+30 pts)	ZEHLA Brain
14	Score sobe para 55 -> cluster muda para WARM	Score Engine
15	Brain dispara: email nurture + alerta vendas	Action Dispatcher

16	Vendedor recebe alerta e entra em contato	WhatsApp + ZCC
17	Lead faz trial -> trial_started (+50 pts)	Funil de vendas
18	Lead vira cliente HOT -> conversao concluida	SMARTHOTEL

Tabela 8: Fluxo completo de campanha ZEHLA Blast

9. ZCC Dashboard — Interface do Blast

O modulo ZEHLA Blast sera integrado ao ZCC (Zehla Central Control) existente como uma nova secao lateral. O dashboard inclui: listagem de campanhas com KPIs em tempo real (enviadas, entregues, leituras, respostas, falhas), grafico de engajamento por campanha, status das instancias WhatsApp (conectadas/desconectadas/banidas), gerenciador de contatos com importacao CSV, editor de templates com preview, e painel de metricas consolidadas do ZEHLA Brain mostrando como as campanhas impactaram o score e cluster dos leads.

Widget	Tipo	Dados
KPI Cards	4 cartoes	Total enviadas, entregues, leituras, respostas
Campaign List	Tabela interativa	Nome, status, progresso, acoes (pause/resume/delete)
Engagement Chart	Grafico de barras	Respostas por campanha (Recharts)
Instance Health	Status badges	Verde (OK), Amarelo (warning), Vermelho (banned)
Contact Manager	Tabela + filtro	Listagem, grupos, import CSV, export
Template Editor	Editor de texto	Variaveis, preview, teste em tempo real
Brain Impact	Cards + mini-chart	Leads WARM/HOT gerados por campanha
Activity Feed	Timeline	Eventos recentes: envio, resposta, opt-out, alerta

Tabela 9: Widgets do Dashboard ZEHLA Blast no ZCC

10. Estrutura de Arquivos — Modulo Blast

Arquivo	Tipo	Descricao
prisma/schema.prisma	MODIFY	Adicionar BlastCampaign, BlastMessage, BlastInstance, BlastContact
lib/evolution-client.ts	NOVO	Cliente REST para Evolution API (envio, QR, status)
lib/blast-queues.ts	NOVO	3 filas BullMQ: blast:send, blast:process, blast:track

services/campaign-sender.ts	NOVO	Lancar campanha, enfileirar mensagens, rotacionar instancias
services/contact-importer.ts	NOVO	Import CSV, higienizar DDI/DDD, deduplicar
services/message-composer.ts	NOVO	Preencher templates com variaveis do contato
services/blast-brain-tracker.ts	NOVO	Tracking de eventos no ZEHLA Brain (LIS)
services/instance-manager.ts	NOVO	Gerenciar pool de instancias, warmup, health check
services/throttler.ts	NOVO	Calculo de delays, lotes, pausas e horarios
app/api/blast/campaigns/route.ts	NOVO	CRUD de campanhas + metricas
app/api/blast/contacts/route.ts	NOVO	CRUD de contatos + import/export
app/api/blast/instances/route.ts	NOVO	Registro e gerenciamento de instancias
app/api/blast/webhook/route.ts	NOVO	Receber respostas do WhatsApp via Evolution API
app/blast/page.tsx	NOVO	Dashboard principal do Blast no ZCC
app/blast/campaigns/[id]/page.tsx	NOVO	Detalhe da campanha com metricas
app/blast/contacts/page.tsx	NOVO	Gerenciador de contatos
app/blast/templates/page.tsx	NOVO	Editor de templates
docker-compose.yml	MODIFY	Adicionar Evolution API + postgres-evo

Tabela 10: Estrutura completa do modulo ZEHLA Blast

11. Varias Tecnicas Recomendadas

Apos a analise dos concorrentes e da documentacao do Evolution API, as seguintes variacoes tecnicas sao recomendadas para maximizar a efetividade e seguranca da ferramenta ZEHLA Blast. Essas variacoes foram identificadas como diferenciais competitivos dos concorrentes e devem ser consideradas na roadmap de implementacao.

Variacao	Prioridade	Descricao
Meta Official API	ALTA	Adicionar suporte a Cloud API da Meta para envio compliant sem risco de banimento. Ideal para escala acima de 1.000 contatos/dia.
Sequencias de Mensagem	ALTA	Funis de mensagem (estilo WaSeller): pre-configurar sequencias de 3-5 msgs que sao disparadas em intervalos ao longo de dias.
Agente IA (SDR-RAG)	MEDIA	Usar o ZEHLA Brain como agente IA de vendas: quando um lead responde, o Brain gera resposta contextual com base na pousada e no historico.
Typebot Integration	MEDIA	Integrar Typebot via Evolution API para criar fluxos conversacionais interativos com opcoes clicaveis (botoes e listas).

Multi-Canal	BAIXA	Expandir para Instagram Direct e Facebook Messenger via Evolution API, centralizando todos os canais no ZCC.
Gravacao de Audio IA	BAIXA	Gerar mensagens de audio com voz sintetica (ElevenLabs/OpenAI TTS) para campanhas de voz, mais pessoais que texto.
A/B Testing	MEDIA	Criar variantes de mensagem e testar qual gera mais respostas. Rastrear qual template converte melhor por segmento.
Warm-up Automatico	ALTA	O sistema automaticamente limita envios baseado na idade da instancia (protocolo da Tabela 5) sem intervencao manual.

Tabela 11: Variacoes tecnicas recomendadas

Vantagem Competitiva do ZEHLA Blast: Diferente do WaSeller (extensao limitada) e do Zap Responder (SaaS terceirizado), o ZEHLA Blast e proprietario, integrado ao ZEHLA Brain (pipeline de 5 estagios com scoring automatico), e usa o Evolution API open-source como base. Isso elimina custos mensais de SaaS, garante controle total dos dados, e fornece inteligencia de vendas que nenhum concorrente oferece: cada resposta de lead automaticamente alimenta o score, classificacao e acoes do ZEHLA Brain, criando um ciclo virtuoso de conversao que vai do primeiro contato a venda fechada.